

Learn Prolog Now!

by Patrick Blackburn, Johan Bos, and Kristina Striegnitz

[LPN! Home](#)

[Free Online Version](#)

[Paperback English](#)

[Paperback Français](#)

[Teaching Prolog](#)

[Prolog Implementations](#)

[Prolog Manuals](#)

[Prolog Links](#)

[Thanks!](#)

[Contact us](#)

[\[next \]](#) [\[prev \]](#) [\[prev-tail \]](#) [\[tail \]](#) [\[up \]](#)

6.3 Exercises

Exercise 6.1 Let's call a list doubled if it is made of two consecutive blocks of elements that are exactly the same. For example, [a,b,c,a,b,c] is doubled (it's made up of [a,b,c] followed by [a,b,c]) and so is [foo,gubble,foo,gubble] . On the other hand, [foo,gubble,foo] is not doubled. Write a predicate doubled(List) which succeeds when List is a doubled list.

Exercise 6.2 A palindrome is a word or phrase that spells the same forwards and backwards. For example, 'rotator', 'eve', and 'nurses run' are all palindromes. Write a predicate palindrome(List) , which checks whether List is a palindrome. For example, to the queries

```
?- palindrome([r,o,t,a,t,o,r]).
```

and

```
?- palindrome([n,u,r,s,e,s,r,u,n]).
```

Prolog should respond yes, but to the query

```
?- palindrome([n,o,t,h,i,s]).
```

it should respond no.

Exercise 6.3 Write a predicate toptail(InList,OutList) which says no if InList is a list containing fewer than 2 elements, and which deletes the first and the last elements of InList and returns the result as OutList , when InList is a list containing at least 2 elements. For example:

```
toptail([a],T).
no
```

```
toptail([a,b],T).
T=[]
```

```
toptail([a,b,c],T).
T=[b]
```

(Hint: here's where append/3 comes in useful.)

Exercise 6.4 Write a predicate last(List,X) which is true only when List is a list that contains at least one element and X is the last element of that list. Do this in two different ways:

1. Define last/2 using the predicate rev/2 discussed in the text.
2. Define last/2 using recursion.

Exercise 6.5 Write a predicate swapfl(List1,List2) which checks whether List1 is identical to List2 , except that the first and last elements are exchanged. Here's where append/3 could come in useful again, but it is

also possible to write a recursive definition without appealing to `append/3` (or any other) predicates.

Exercise 6.6 Here is an exercise for those of you who like logic puzzles.

There is a street with three neighbouring houses that all have a different colour, namely red, blue, and green. People of different nationalities live in the different houses and they all have a different pet. Here are some more facts about them:

- The Englishman lives in the red house.
- The jaguar is the pet of the Spanish family.
- The Japanese lives to the right of the snail keeper.
- The snail keeper lives to the left of the blue house.

Who keeps the zebra? Don't work it out for yourself: define a predicate `zebra/1` that tells you the nationality of the owner of the zebra!

(Hint: Think of a representation for the houses and the street. Code the four constraints in Prolog. You may find `member/2` and `sublist/2` useful.)

[[next](#)] [[prev](#)] [[prev-tail](#)] [[front](#)] [[up](#)]

